

**PROJECT DOCUMENTATION**

# **BUYIT – IN YOUR WAY**

*An E-Commerce Web Application*

Developed as an Internship Project under the guidance of

**INTERNbee**

Submitted by

**Name: Amit Kumar**

**Registration No.: 1341825037**

**Name: Shruchi Saugandh**

**Registration No.: 1341825601**

**Semester: 3<sup>rd</sup>**

**Branch: Computer Science and Engineering**

**Government Polytechnic, Raghapur**

Supaul, Bihar

Version 1.0 – 2026

## DECLARATION

---

We hereby declare that the project work entitled "**Buyit – In Your Way**" submitted to Government Polytechnic, Raghapur, Supaul, is a record of original work done by us under the guidance of INTERNbee. This work has not been submitted to any other institution or organisation for any other purpose.

All external resources, libraries and services used in this project have been acknowledged in the references section.

Amit Kumar  
Shruchi Saugandh

## ACKNOWLEDGEMENT

---

We take this opportunity to express our sincere gratitude to everyone who supported us during the development of this project.

We are thankful to **INTERNbee** for giving us the opportunity to work on a practical project and for the guidance and learning resources provided during the internship. The experience helped us to understand how a real web application is planned, built and tested.

We express our heartfelt thanks to the Principal, the Head of Department and all the faculty members of **Government Polytechnic, Raghapur, Supaul** for their encouragement and for the strong foundation of knowledge they have given us.

We also thank our families and friends for their constant support and motivation, and the creators of the free tools and services – Firebase, EmailJS, Font Awesome and Google Fonts – that made this project possible.

Amit Kumar  
Shruchi Saugandh

## ABSTRACT

---

Online shopping has changed the way people buy products, but building an e-commerce platform normally requires servers, databases and significant cost. **Buyit – In Your Way** is a frontend e-commerce web application developed to show that a complete shopping experience can be created using standard web technologies and freely available services.

The application is built with HTML5, CSS3 and vanilla JavaScript. It offers a home page with categories and featured products, a product listing with real-time search, detailed product pages, a persistent shopping cart, a Buy Now option, and a three-step checkout with validation of address and payment details. User registration and login, including Google sign-in, are handled by Firebase Authentication. After an order is placed, a confirmation email is sent through EmailJS. Orders can be viewed and cancelled from an order history page. Cart and order data are kept in the browser using localStorage and sessionStorage.

The project follows a client-side architecture with separate JavaScript modules for authentication, cart management and checkout, and a single stylesheet that gives a consistent and responsive design. This document describes the analysis, design, implementation, testing, security considerations and future scope of the project. Buyit is a functional prototype whose main purpose is learning; it provides a base that can be extended with a database, a payment gateway and an administrator panel.

**Keywords:** E-commerce, HTML, CSS, JavaScript, Firebase Authentication, EmailJS, localStorage, Responsive Design

# TABLE OF CONTENTS

---

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>1. Introduction.....</b>                                | <b>8</b>  |
| 1.1 Background.....                                        | 8         |
| 1.2 About the Project.....                                 | 8         |
| 1.3 Problem Statement.....                                 | 8         |
| 1.4 Objectives.....                                        | 8         |
| 1.5 Scope of the Project.....                              | 9         |
| 1.6 About the Internship.....                              | 9         |
| 1.7 Project Team.....                                      | 9         |
| 1.8 Organization of this Document.....                     | 10        |
| <b>2. System Analysis.....</b>                             | <b>10</b> |
| 2.1 Existing System.....                                   | 10        |
| 2.2 Proposed System.....                                   | 10        |
| 2.3 Feasibility Study.....                                 | 11        |
| 2.4 Requirement Analysis.....                              | 11        |
| 2.5 Hardware and Software Requirements.....                | 12        |
| 2.6 Users of the System.....                               | 13        |
| <b>3. Technology Stack.....</b>                            | <b>13</b> |
| 3.1 Technology Summary.....                                | 13        |
| 3.2 HTML5.....                                             | 13        |
| 3.3 CSS3.....                                              | 13        |
| 3.4 JavaScript.....                                        | 14        |
| 3.5 Firebase Authentication.....                           | 14        |
| 3.6 EmailJS.....                                           | 14        |
| 3.7 Web Storage API.....                                   | 14        |
| 3.8 Supporting Tools.....                                  | 14        |
| 3.9 Why No Framework Was Used.....                         | 14        |
| <b>4. System Design.....</b>                               | <b>14</b> |
| 4.1 System Architecture.....                               | 15        |
| 4.2 Project Folder Structure.....                          | 15        |
| 4.3 Pages and Navigation.....                              | 16        |
| 4.4 Data Design.....                                       | 16        |
| 4.5 Data Flow.....                                         | 17        |
| 4.6 Use Case Description.....                              | 17        |
| 4.7 User Flow.....                                         | 18        |
| <b>5. Implementation.....</b>                              | <b>18</b> |
| 5.1 Home Page (index.html).....                            | 18        |
| 5.2 Products Page (products.html).....                     | 18        |
| 5.3 Product Details Page (product-details.html).....       | 19        |
| 5.4 Login Page (login.html).....                           | 19        |
| 5.5 Cart Page (cart.html).....                             | 19        |
| 5.6 Checkout Page (checkout.html).....                     | 19        |
| 5.7 Order Confirmation Page (order-confirmation.html)..... | 20        |
| 5.8 My Orders Page (orders.html).....                      | 20        |
| 5.9 Account Page (account.html).....                       | 20        |
| 5.10 JavaScript Modules.....                               | 20        |
| <b>6. Authentication Using Firebase.....</b>               | <b>22</b> |
| 6.1 Why Firebase Authentication.....                       | 22        |
| 6.2 Authentication Methods.....                            | 22        |
| 6.3 Login Flow.....                                        | 22        |

|                                              |           |
|----------------------------------------------|-----------|
| 6.4 Google Sign-In.....                      | 23        |
| 6.5 Session Persistence.....                 | 23        |
| 6.6 Protected Pages.....                     | 23        |
| 6.7 Error Messages.....                      | 23        |
| <b>7. Email Notification System.....</b>     | <b>23</b> |
| 7.1 About EmailJS.....                       | 23        |
| 7.2 How It Works in Buyit.....               | 23        |
| 7.3 Template Variables.....                  | 24        |
| 7.4 Handling Failures.....                   | 24        |
| <b>8. User Interface Design.....</b>         | <b>24</b> |
| 8.1 Colour Palette.....                      | 24        |
| 8.2 Typography.....                          | 25        |
| 8.3 Layout Techniques.....                   | 25        |
| 8.4 Responsive Design.....                   | 25        |
| 8.5 Interface Feedback.....                  | 25        |
| <b>9. Testing.....</b>                       | <b>25</b> |
| 9.1 Testing Approach.....                    | 25        |
| 9.2 Test Cases.....                          | 25        |
| 9.3 Observations.....                        | 27        |
| <b>10. Security Considerations.....</b>      | <b>27</b> |
| 10.1 What Is Handled Securely.....           | 27        |
| 10.2 Points to Improve.....                  | 27        |
| <b>11. Limitations and Future Scope.....</b> | <b>28</b> |
| 11.1 Current Limitations.....                | 28        |
| 11.2 Future Scope.....                       | 28        |
| <b>12. Conclusion.....</b>                   | <b>28</b> |
| 12.1 Summary.....                            | 28        |
| 12.2 Learning Outcomes.....                  | 29        |
| 12.3 Team Collaboration.....                 | 29        |
| 12.4 Acknowledgement of Support.....         | 29        |
| <b>References.....</b>                       | <b>29</b> |
| <b>Appendix A: Running the Project.....</b>  | <b>30</b> |
| A.1 Running Locally.....                     | 30        |
| A.2 Setting Up Firebase.....                 | 30        |
| A.3 Setting Up EmailJS.....                  | 30        |
| A.4 Deployment.....                          | 30        |
| <b>Appendix B: Glossary.....</b>             | <b>30</b> |

# 1. Introduction

---

## 1.1 Background

Online shopping has become part of everyday life in India. Improved internet access, low-cost smartphones and digital payment options such as UPI have made it possible for people in small towns and large cities alike to buy products without visiting a physical shop. A customer today expects to search for a product, compare its price and specifications, place an order from a phone or laptop and receive a confirmation within minutes.

For a student of computer science, an e-commerce website is an ideal project because it brings together almost every core idea of web development in one application: page design, navigation, forms, data validation, user accounts, state management, and communication with external services. Building such a system from scratch, rather than using a ready-made template, gives a much deeper understanding of how real websites work behind the screen.

## 1.2 About the Project

**Buyit – In Your Way** is a frontend e-commerce web application developed using HTML5, CSS3 and vanilla JavaScript. It allows a visitor to browse products by category, search for a product by name, view detailed product information, add products to a shopping cart, and place an order through a three-step checkout process. Registered users can log in using an email address and password or with their Google account, view their past orders, and cancel orders that have not yet been delivered. After an order is placed, the system sends an automatic confirmation email to the customer.

The project does not use any frontend framework or a dedicated backend server. User authentication is handled by Google Firebase Authentication, order confirmation emails are sent through the EmailJS service, and the shopping cart and order history are stored in the browser using the Web Storage API. This keeps the project lightweight and makes it possible to host it on any static hosting platform.

## 1.3 Problem Statement

Small and medium sellers often lack a simple, low-cost way to present their products online. Building a full e-commerce platform requires a backend server, a database, a payment gateway and continuous maintenance, which is expensive and technically demanding for a beginner or a small business. At the same time, a student learning web development needs a realistic project in which every part of the shopping journey can be built, understood and tested.

The problem addressed by this project is therefore to design and develop a functional, user-friendly online shopping website that covers the complete customer journey – from product discovery to order confirmation – using only client-side technologies and freely available third-party services.

## 1.4 Objectives

The main objectives of the Buyit project are:

- To design a clean and consistent user interface for an online store that works on desktop and mobile screens.
- To implement product browsing by category and real-time search by product name.
- To provide detailed product pages with price, offers, options and specifications.
- To build a shopping cart that remembers its contents even after the page is refreshed or the browser is closed.
- To integrate secure user registration and login using Firebase Authentication, including Google sign-in.

- To implement a multi-step checkout with proper validation of address and payment information.
- To send an automatic order confirmation email to the customer using EmailJS.
- To maintain an order history in which a customer can see the status of every order and cancel an eligible order.
- To learn and apply the fundamentals of HTML, CSS, JavaScript, browser storage and third-party API integration through a practical project.

## 1.5 Scope of the Project

The scope of version 1.0 of Buyit covers the following areas:

- A public storefront where anyone can browse products, search and add items to the cart without logging in.
- Protected pages (checkout, orders, order confirmation and account) which require the user to be logged in.
- Three payment options at checkout – Cash on Delivery, UPI and Credit/Debit Card – presented as a demonstration of the payment step.
- Order history and cancellation for the logged-in user on the same browser.
- Email notification at the time of order placement.

The following items are outside the scope of this version and are described as future work in Chapter 11: real payment processing, an administrator panel, a product database, automatic order status updates, and storing orders in a cloud database so that they are available on every device.

## 1.6 About the Internship

This project was developed as a second-year internship project under the guidance of **INTERNbee**, an internship and training platform that gives students practical exposure to real development tasks. The internship was undertaken by students of **Government Polytechnic, Raghopur, Supaul (Bihar)**. The aim of the internship was to move from classroom theory to a working product: planning a project, dividing the work between team members, writing and testing code, using third-party services, and preparing documentation.

During the internship the team learned how a web application is structured into pages, styles and scripts; how data can be kept in the browser; how authentication works without writing server code; and how to prepare a project so that others can understand, run and extend it.

## 1.7 Project Team

**Table 1: Project team details**

| Item                | Details                                          |
|---------------------|--------------------------------------------------|
| Project Title       | Buyit – In Your Way (E-Commerce Web Application) |
| Team Members        | Amit Kumar, Shruchi Saugandh                     |
| Institution         | Government Polytechnic, Raghopur, Supaul, Bihar  |
| Internship Provider | INTERNbee                                        |
| Project Type        | Second-year internship project (team of two)     |
| Version             | 1.0 – Functional Prototype                       |

## 1.8 Organization of this Document

This document is organised into twelve chapters followed by references and appendices. Table 2 gives a short guide to what each chapter contains.

**Table 2: Chapter overview**

| Chapter | Title                        | Content                                                 |
|---------|------------------------------|---------------------------------------------------------|
| 1       | Introduction                 | Background, problem, objectives, scope and team         |
| 2       | System Analysis              | Existing and proposed system, feasibility, requirements |
| 3       | Technology Stack             | Languages, services and tools used and why              |
| 4       | System Design                | Architecture, folder structure, data design, user flows |
| 5       | Implementation               | Page-by-page and module-by-module description           |
| 6       | Authentication               | Firebase Authentication and session handling            |
| 7       | Email Notification           | EmailJS integration and template variables              |
| 8       | User Interface Design        | Colours, typography, layout and responsiveness          |
| 9       | Testing                      | Test approach and test cases                            |
| 10      | Security Considerations      | Protections in place and recommended improvements       |
| 11      | Limitations and Future Scope | Known limitations and planned enhancements              |
| 12      | Conclusion                   | Learning outcomes and final remarks                     |

## 2. System Analysis

---

System analysis is the study of the problem before any code is written. It identifies what the users need, what the existing alternatives offer, whether the project is practical to build, and what the system must do and what resources it needs.

### 2.1 Existing System

Customers currently buy products in two main ways. In the traditional method, a customer visits a local shop, looks at the products physically, negotiates the price and pays on the spot. This method is limited by shop timings, travel effort and the small range of products available in a single shop. Many small sellers also have no online presence at all, so people outside their locality cannot find them.

The second method is to use large e-commerce platforms. These platforms are powerful but complex: they depend on large backend systems, logistics networks and payment gateways. A seller using them must follow their rules, pay commissions and cannot easily customise the look and flow of the store. For a student, such platforms are also a black box – it is not possible to see how they work internally.

### 2.2 Proposed System

Buyit proposes a lightweight online store that can be understood, run and hosted with very little cost. The proposed system consists of a set of linked web pages that provide the complete shopping flow. It uses Firebase for login so that password handling is done securely by Google, EmailJS for order emails so that no mail server is required, and the browser's own storage for the cart and orders so that no database server is required.

The advantages of the proposed system are:

- No server setup or maintenance is needed; the project can be opened directly in a browser or hosted on a static host.
- Secure login without writing password-handling code.
- A consistent design across all pages through one shared stylesheet.
- Persistent cart across page refreshes and browser restarts.
- A clear and easily extendable code structure that is suitable for learning and for future upgrades.

## 2.3 Feasibility Study

A feasibility study checks whether the project is practical from several points of view.

**Table 3: Feasibility analysis**

| Type        | Assessment                                                                                                                                                                                                                                     |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Technical   | All required technologies (HTML, CSS, JavaScript, Firebase Auth, EmailJS) are well documented, run in any modern browser and were within the skills the team could learn during the internship. Feasible.                                      |
| Economic    | Every tool and service used has a free plan sufficient for a prototype: Firebase Authentication, EmailJS (200 emails per month), Google Fonts, Font Awesome and free static hosting. No development cost other than the team's time. Feasible. |
| Operational | The interface follows familiar online-shopping conventions (search bar, cart icon, product cards, step-by-step checkout), so users need no training. Feasible.                                                                                 |
| Schedule    | The project was divided into pages and modules which could be developed in parallel by two members and completed within the internship period. Feasible.                                                                                       |

## 2.4 Requirement Analysis

### 2.4.1 Functional Requirements

**Table 4: Functional requirements**

| ID   | Requirement                                                                                                          |
|------|----------------------------------------------------------------------------------------------------------------------|
| FR1  | The system shall display a home page with a promotional banner, category cards and featured products.                |
| FR2  | The system shall list products grouped by category on a products page.                                               |
| FR3  | The system shall allow the user to search products by name from any page.                                            |
| FR4  | The system shall show a detail page for each product with price, description, offers, options and specifications.    |
| FR5  | The system shall allow adding a product to the cart without logging in.                                              |
| FR6  | The system shall allow changing the quantity of a cart item and removing it, and shall update the total immediately. |
| FR7  | The system shall provide a "Buy Now" option that sends a single product directly to checkout.                        |
| FR8  | The system shall allow a user to register and log in using email and password and using a Google account.            |
| FR9  | The system shall restrict checkout, orders, order confirmation and account pages to logged-in users.                 |
| FR10 | The system shall collect and validate delivery address and payment preference in a multi-step checkout.              |
| FR11 | The system shall save each placed order, clear the cart, and display an order confirmation page.                     |

| ID   | Requirement                                                                                                                          |
|------|--------------------------------------------------------------------------------------------------------------------------------------|
| FR12 | The system shall send an order confirmation email to the logged-in user.                                                             |
| FR13 | The system shall display the order history with status and allow cancellation of orders that are not delivered or already cancelled. |
| FR14 | The system shall allow the user to log out.                                                                                          |

## 2.4.2 Non-Functional Requirements

**Table 5: Non-functional requirements**

| Attribute       | Requirement                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------------------|
| Usability       | Simple navigation, clear buttons, readable fonts, friendly error messages instead of technical codes. |
| Responsiveness  | Layout must adapt to desktop, tablet and mobile screen widths.                                        |
| Performance     | Pages load quickly because there is no build step, no heavy framework and only a few images.          |
| Reliability     | A failure in sending the confirmation email must not stop an order from being placed.                 |
| Maintainability | Shared styling in one CSS file and separate JavaScript modules for authentication, cart and checkout. |
| Portability     | Runs in any modern browser and on any static hosting service.                                         |
| Security        | Passwords are never handled by the project code; authentication is delegated to Firebase.             |

## 2.5 Hardware and Software Requirements

**Table 6: Development environment**

| Component        | Requirement                                                                |
|------------------|----------------------------------------------------------------------------|
| Processor        | Any dual-core processor or better                                          |
| Memory           | 4 GB RAM or more                                                           |
| Operating System | Windows, Linux or macOS                                                    |
| Code Editor      | Visual Studio Code (or any text editor)                                    |
| Browser          | Google Chrome, Microsoft Edge, Mozilla Firefox or Safari (latest versions) |
| Version Control  | Git and GitHub                                                             |
| Internet         | Required for Firebase, EmailJS, fonts and icon libraries                   |

**Table 7: User-side requirements**

| Component | Requirement                                                  |
|-----------|--------------------------------------------------------------|
| Device    | Smartphone, tablet, laptop or desktop                        |
| Browser   | Any modern browser with JavaScript and local storage enabled |
| Internet  | Active internet connection for login and email features      |

## 2.6 Users of the System

**Table 8: System users**

| User                | Description                             | Access                                                                           |
|---------------------|-----------------------------------------|----------------------------------------------------------------------------------|
| Visitor             | A person who has not logged in          | Home, products, product details, search, cart, login page                        |
| Registered customer | A person logged in with email or Google | All visitor features plus checkout, order confirmation, orders and account pages |

## 3. Technology Stack

The technologies for Buyit were selected with three goals: keep the project simple enough to be built by a two-member student team, keep it free of cost, and make sure the team learns the fundamentals rather than depending on a large framework. This chapter lists the technologies and explains the role of each one.

### 3.1 Technology Summary

**Table 9: Technologies used in Buyit**

| Technology              | Version / Source     | Purpose                                         |
|-------------------------|----------------------|-------------------------------------------------|
| HTML5                   | Native browser       | Structure and content of every page             |
| CSS3                    | Native browser       | Styling, layout and responsive design           |
| JavaScript              | ES6+, native browser | Cart logic, search, validation, page behaviour  |
| Firebase Authentication | v10.8.0 (Google CDN) | User registration, login and Google sign-in     |
| EmailJS                 | v4 (jsDelivr CDN)    | Order confirmation emails sent from the browser |
| Font Awesome            | v6 (CDN)             | Icons for cart, user, status and buttons        |
| Google Fonts            | Inter family         | Typography across the website                   |
| localStorage            | Browser API          | Persistent storage of cart and orders           |
| sessionStorage          | Browser API          | Temporary storage of the last placed order      |

### 3.2 HTML5

HTML (HyperText Markup Language) defines the content and structure of each page. Buyit uses semantic elements such as header, nav, section and footer, forms with input types suited to the data being entered, and id attributes that allow scripts and links to reach specific parts of a page. Nine HTML pages make up the application, each serving one purpose in the shopping journey.

### 3.3 CSS3

CSS (Cascading Style Sheets) controls the appearance of the pages. The whole project is styled by one file, style.css, of nearly 900 lines. It uses CSS custom properties (variables) for the colour scheme, Flexbox and Grid for layouts, the :target selector to show the selected product on the details page, transitions for hover and toast effects, and media queries for mobile responsiveness.

### 3.4 JavaScript

JavaScript provides all the dynamic behaviour. The project uses plain ES6+ JavaScript without any framework. This choice was deliberate: handling the Document Object Model (DOM), events, JSON and browser storage directly gives a clear understanding of what frameworks do internally. The code is divided into three files – `script.js`, `auth.js` and `checkout.js` – each responsible for a separate part of the application.

### 3.5 Firebase Authentication

Firebase is a platform from Google that provides ready-made backend services. Buyit uses only its Authentication service. It stores user accounts securely, verifies passwords, supports Google sign-in and keeps the user logged in across visits. This removes the need to write and host server code for login.

### 3.6 EmailJS

EmailJS allows emails to be sent directly from client-side JavaScript. The developer designs an email template on the EmailJS dashboard with placeholders; the application sends the values of those placeholders, and EmailJS delivers the finished email through a connected email service. This made it possible to send branded order confirmation emails without running a mail server.

### 3.7 Web Storage API

The Web Storage API lets a website store key–value data inside the user's browser. `localStorage` keeps data until it is deleted and is used for the cart and the order history. `sessionStorage` keeps data only until the tab is closed and is used to pass the just-placed order from the checkout page to the confirmation page.

### 3.8 Supporting Tools

- **Visual Studio Code** – writing and editing code.
- **Git and GitHub** – version control and sharing of code between the two team members.
- **Browser developer tools** – inspecting elements, debugging JavaScript, checking storage contents and testing mobile layouts.
- **Font Awesome and Google Fonts** – icons and typography loaded from CDNs.

### 3.9 Why No Framework Was Used

Frameworks such as React, Angular or Vue are valuable in large projects, but they hide many basic operations. The team chose to first learn how the DOM is updated, how data is serialised to JSON, how events are handled and how external APIs are called. The result is a project with no build step and no dependencies to install, which can be opened in a browser directly. This foundation makes it easier to adopt a framework in the future.

## 4. System Design

---

System design converts the requirements into a plan for how the application is organised. This chapter describes the architecture, the project structure, the pages and their relationships, the way data is stored and the flow of a user through the system.

## 4.1 System Architecture

Buyit follows a client-side architecture. All the application logic runs in the user's browser. External services are used only for tasks that cannot be safely done in the browser alone, namely authentication and email delivery. The architecture can be understood as three layers.

**Table 10: Layers of the Buyit architecture**

| Layer                   | Components                                           | Responsibility                                                   |
|-------------------------|------------------------------------------------------|------------------------------------------------------------------|
| Presentation layer      | HTML pages, style.css                                | Displays products, forms, cart and orders to the user            |
| Application logic layer | script.js, auth.js, checkout.js                      | Cart operations, search, validation, login state, order creation |
| Data and service layer  | localStorage, sessionStorage, Firebase Auth, EmailJS | Keeps cart and orders, verifies users, sends emails              |

When the user performs an action – for example clicking "Add to Cart" – the event is handled by a JavaScript function in the logic layer, which updates the data in browser storage and then updates what is shown on the page. For login, the logic layer calls Firebase and reacts to the result. For placing an order, the logic layer saves the order in the browser and calls EmailJS to send the confirmation email.

## 4.2 Project Folder Structure

All files are kept in one root folder. There is no build process; the browser reads the files directly.

```
Buyit-main/
|-- index.html           Home page
|-- products.html       Product listing and search results
|-- product-details.html Details of every product
|-- login.html          Login and registration
|-- cart.html           Shopping cart
|-- checkout.html       Three-step checkout
|-- order-confirmation.html Post-purchase summary
|-- orders.html         Order history and cancellation
|-- account.html        Logged-in user account page
|-- css/
|   |-- style.css       Single stylesheet for all pages
|-- js/
|   |-- auth.js         Firebase authentication (ES module)
|   |-- script.js       Cart, search, Buy Now, toast
|   |-- checkout.js     Checkout flow and EmailJS (ES module)
|-- images/             Banner and product images
|-- README.md           Project overview
`-- Buyit-Documentations.* Earlier documentation (docx, pdf)
```

JavaScript files that import Firebase (auth.js and checkout.js) are loaded with type="module". The file script.js does not use imports and is loaded as a normal script; its functions are attached to the window object so that onclick attributes in the HTML can call them.

## 4.3 Pages and Navigation

**Table 11: Pages of the application**

| Page                    | Purpose                                                        | Login required |
|-------------------------|----------------------------------------------------------------|----------------|
| index.html              | Landing page with banner, category cards and featured products | No             |
| products.html           | All products grouped by category; search results               | No             |
| product-details.html    | Full details of one product, selected by URL hash              | No             |
| cart.html               | Review cart items and total                                    | No             |
| login.html              | Login, registration and Google sign-in                         | No             |
| checkout.html           | Address, payment and review steps                              | Yes            |
| order-confirmation.html | Shows the order just placed                                    | Yes            |
| orders.html             | Order history and cancellation                                 | Yes            |
| account.html            | Shows user details and links; logout                           | Yes            |

## 4.4 Data Design

Because there is no database, data is organised as JSON objects stored in browser storage under fixed keys. The table below lists the storage keys, and the following sub-sections show the structure of the stored objects.

**Table 12: Browser storage keys**

| Key              | Storage        | Content                        | Lifetime                                 |
|------------------|----------------|--------------------------------|------------------------------------------|
| buyit_cart       | localStorage   | Array of cart items            | Until order is placed or data is cleared |
| buyit_orders     | localStorage   | Array of all placed orders     | Until browser data is cleared            |
| buyit_last_order | sessionStorage | The most recently placed order | Until the tab is closed                  |

### 4.4.1 Cart Item Structure

```
{
  id:      "motorola-edge",
  name:    "MOTOROLA Edge 60 Fusion 5G",
  price:   29999,
  image:   "images/mobile-image.jpg",
  quantity: 2
}
```

### 4.4.2 Order Structure

```
{
  orderId:      "BYT12345678",
  orderDate:    "24 June 2026",
  deliveryDate: "1 July 2026",
  address:      "12 MG Road, Patna, Bihar - 800001",
  paymentMethod: "Cash on Delivery",
  items:        [ ...cart items ],
  total:        29999,
  status:       "Placed"
}
```

```
}
```

### 4.4.3 Product Catalogue

Products are written directly into the HTML pages. The store contains five products, each identified by an id that is used consistently in the product listing, the details page URL hash and the cart.

**Table 13: Products available in the store**

| Product                    | Category | Price (INR) | Detail page id |
|----------------------------|----------|-------------|----------------|
| MOTOROLA Edge 60 Fusion 5G | Mobiles  | 29,999      | motorola-edge  |
| Samsung Galaxy F70e 5G     | Mobiles  | 13,499      | samsung-f70e   |
| vivo X200T                 | Mobiles  | 64,999      | vivo-x200t     |
| Lenovo IdeaPad Slim 3      | Laptops  | 69,999      | lenovo-ideapad |
| Cotton Graphic T-Shirt     | Clothes  | 799         | mens-tshirt    |

## 4.5 Data Flow

The data moves through the application in the following stages.

- **Browsing:** product information is read from the HTML page itself; nothing is fetched from a server.
- **Cart:** when "Add to Cart" is clicked, the product data is read from the button's parameters, added to the cart array in memory and written to localStorage as JSON. The cart page reads the same key to draw the cart.
- **Authentication:** credentials entered on the login page are sent to Firebase. Firebase returns a user object and keeps the session. The listener in auth.js reads the session on every page.
- **Order:** at checkout the cart is read from localStorage, the form values are read from the page, an order object is created and stored in localStorage and sessionStorage, and the cart key is removed.
- **Email:** the order values are sent to EmailJS, which fills the template and sends the email to the user.

## 4.6 Use Case Description

**Table 14: Main use cases**

| Use case                   | Actor             | Description                                                |
|----------------------------|-------------------|------------------------------------------------------------|
| Browse and search products | Visitor, Customer | View categories and product cards, search by name          |
| View product details       | Visitor, Customer | Open a product page and read its specifications and offers |
| Manage cart                | Visitor, Customer | Add items, change quantity, remove items                   |
| Register / Log in          | Visitor           | Create an account or log in with email or Google           |
| Place order                | Customer          | Enter address, choose payment, review and confirm          |
| View orders                | Customer          | See all past orders with their status                      |
| Cancel order               | Customer          | Cancel an order that is not delivered or cancelled         |
| Log out                    | Customer          | End the session                                            |

## 4.7 User Flow

### *Path 1 – Browsing and buying*

- The user opens index.html and sees the banner, categories and featured products.
- The user goes to products.html and opens a product to read its details.
- The user clicks "Add to Cart"; the item is stored and a toast message appears.
- The user opens cart.html, checks the items and clicks "Proceed to Checkout".
- If the user is not logged in, auth.js redirects to login.html; after logging in the user returns to the home page and proceeds to checkout again.
- The user completes the address, payment and review steps and clicks "Place Order".
- The order is saved, the cart is emptied, the confirmation email is requested and order-confirmation.html is shown.

### *Path 2 – Buy Now*

On a product details page the user clicks "Buy Now". The cart is cleared, the selected product is added with quantity one and the user is taken straight to checkout.html, skipping the cart page.

### *Path 3 – Cancelling an order*

The user opens orders.html from the account page, finds an order that is Placed, Processing or Shipped, clicks "Cancel Order" and confirms in the pop-up. The status is changed to Cancelled in localStorage and the page is redrawn immediately.

## 5. Implementation

---

This chapter explains how each page and JavaScript module was implemented. The description follows the order in which a customer meets the pages.

### 5.1 Home Page (index.html)

The home page has a navigation bar, a promotional banner, a "Shop by Category" section and a "Featured Products" section. The navigation bar contains the Buyit logo, a search form, links to Home, Shop and Orders, a cart icon and a login button which is replaced by the username after login. The category grid contains seven cards: Mobiles, Laptops, Clothes, Shoes, Watches, Headphones and Books. Each card links to the matching section of the products page using an anchor such as products.html#mobiles. Products are currently available for four of these categories; the remaining categories are placeholders for future stock.

Featured products are shown as cards with an image, name, price and an "Add to Cart" button that works immediately without login. A "See more" button leads to the full products page.

### 5.2 Products Page (products.html)

The products page lists all products under category headings. Each category is a section with its own id so that links from the home page scroll to the right place. Products are displayed in a CSS Grid that automatically fits as many columns as the screen allows with a minimum card width of 250 pixels.

When the page is opened with a search term in the URL (for example products.html?q=lenovo) the function executeSearch() hides the category cards, makes every product section visible, and shows only the product cards whose heading contains the search text. If no card matches, a message "No results

found" is added to the grid. The search is not case sensitive because both the heading and the query are converted to lower case before comparison.

### 5.3 Product Details Page (product-details.html)

All five product pages are stored in one HTML file. Each product is a section with a unique id. By default every section is hidden by CSS; the `:target` selector displays only the section whose id matches the hash in the URL. For example, `product-details.html#lenovo-ideapad` shows only the Lenovo section. This technique allows several pages' worth of content to be kept in one file without JavaScript routing.

Each detail section contains the product image, name, rating, price, description, offers, option selectors such as colour or storage, a specifications table and two buttons. "Add to Cart" calls `addToCart()`. "Buy Now" calls `buyNow()`, which empties the cart, adds the selected product with quantity one and sends the user to the checkout page.

### 5.4 Login Page (login.html)

The login page provides one form for both new and existing users. The Login button submits the form and calls `Firebase signInWithEmailAndPassword`. The Register button calls `createUserWithEmailAndPassword`. A "Sign in with Google" button calls `signInWithPopup` with a Google provider. Errors returned by Firebase are mapped to simple sentences by the function `friendlyError()`, for example "Incorrect password. Please try again." In case a user who is already logged in opens this page, the authentication listener sends them to the home page.

### 5.5 Cart Page (cart.html)

The cart page reads the `buyit_cart` key and draws each item with its image, name, price, quantity control and Remove button. The plus and minus buttons call `updateQuantity()`; when the quantity falls to zero the item is removed. The order summary shows the subtotal and total, and delivery is displayed as free. If the cart is empty, a friendly message with a "Start Shopping" button is shown and the checkout button is disabled.

A small toast message slides up from the bottom of the screen when a product is added, replacing the browser's alert box. It disappears by itself after 2.5 seconds, which keeps the shopping experience smooth.

### 5.6 Checkout Page (checkout.html)

The checkout is a three-step process on a single page. A progress bar at the top shows the active step. Only one step panel is visible at a time; the others carry a "hidden" class. A summary of the cart is always displayed on the right-hand side.

**Table 15: Checkout steps and validation**

| Step                | Content                                                            | Validation rules                                                                                                       |
|---------------------|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 1. Delivery Address | Full name, phone, street address, city, state (dropdown), PIN code | No field may be empty; phone must be exactly 10 digits; PIN must be exactly 6 digits                                   |
| 2. Payment Method   | Cash on Delivery, UPI or Credit/Debit Card                         | UPI ID must contain "@"; card needs 16 digits, expiry of 5 characters and CVV of at least 3 digits; COD needs no input |
| 3. Review and Place | Address, payment method and item list                              | Cart must not be empty                                                                                                 |

The function `goToStep()` controls movement between steps. Before showing step 2 it calls `validateStep1()`, and before step 3 it calls `validateStep2()`. If validation fails, the faulty field is

highlighted with an error style and a message tells the user what to correct. Card numbers are automatically spaced while typing for readability.

## 5.7 Order Confirmation Page (order-confirmation.html)

After the order is placed the user is sent to this page. It reads the order from sessionStorage (key `buyit_last_order`) and displays the order number, order date, expected delivery date, payment method, delivery address, items with quantities and the total amount. An animated tick mark confirms success. If the page is opened directly and no order is found, a message "No recent order found" is displayed instead.

## 5.8 My Orders Page (orders.html)

The orders page reads `buyit_orders` from localStorage and displays the orders with the newest first. Each order card shows the order ID, date, a coloured status badge, items, delivery details and the total. An order that is not Delivered and not Cancelled shows a "Cancel Order" button. The button opens a confirmation pop-up; if the user confirms, the status is changed to Cancelled, the data is saved and the list is drawn again.

**Table 16: Order statuses**

| Status     | Badge colour | Meaning                                |
|------------|--------------|----------------------------------------|
| Placed     | Blue         | Order has been placed (default status) |
| Processing | Orange       | Order is being prepared                |
| Shipped    | Green        | Order is on the way                    |
| Delivered  | Green        | Order has reached the customer         |
| Cancelled  | Red          | Order was cancelled by the user        |

In version 1.0 every new order begins with the status Placed. Because there is no backend, the other statuses are supported by the interface but are not changed automatically.

## 5.9 Account Page (account.html)

The account page is available only to logged-in users. It displays the username (the part of the email before the @ sign), the full email address and links to My Orders, My Cart and Checkout. The Logout button signs the user out of Firebase and returns to the home page.

## 5.10 JavaScript Modules

### 5.10.1 *script.js – cart, search and toast*

This file is loaded as a normal script on pages that need cart or search features. It keeps the cart in a variable initialised from localStorage and saves it after every change.

**Table 17: Functions in script.js**

| Function                                       | Description                                                                                           |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>addToCart(id, name, price, image)</code> | Adds a product or increases its quantity; cleans the price to a number; saves the cart; shows a toast |
| <code>removeFromCart(id)</code>                | Removes an item by id and redraws the cart                                                            |
| <code>updateQuantity(id, qty)</code>           | Changes quantity; removes the item if quantity becomes zero or less                                   |
| <code>saveCart()</code>                        | Stores the cart array as JSON in localStorage                                                         |

| Function                       | Description                                                      |
|--------------------------------|------------------------------------------------------------------|
| renderCart()                   | Draws the cart items and totals on the cart page                 |
| buyNow(id, name, price, image) | Replaces the cart with one item and opens checkout               |
| showCartToast(name)            | Displays the slide-up "added to cart" message                    |
| executeSearch(query)           | Filters product cards by name and handles the no-results message |

The core of the cart is the addToCart function, shown below.

```
function addToCart(id, name, price, image) {
  const existingItem = cart.find(item => item.id === id);
  if (existingItem) {
    existingItem.quantity += 1;
  } else {
    const cleanPrice = parseFloat(String(price).replace(/^[^0-9.-]+/g, ""));
    cart.push({ id, name, price: cleanPrice, image, quantity: 1 });
  }
  saveCart();
  showCartToast(name);
}
```

The regular expression removes the rupee symbol and commas from a price such as "₹29,999" so that it can be stored as the number 29999 and used for calculations.

### 5.10.2 auth.js – authentication

This ES module initialises Firebase, handles login, registration, Google sign-in and logout, and contains the authentication state listener. It is included on every page so the navigation bar always shows the correct state. The listener also protects the pages account.html, checkout.html, orders.html and order-confirmation.html by redirecting visitors who are not logged in to login.html.

```
onAuthStateChanged(auth, (user) => {
  if (user) {
    // hide Login button, show username
  } else {
    // show Login button
    // if current page is protected -> redirect to login.html
  }
});
```

### 5.10.3 checkout.js – checkout and email

This ES module controls the checkout flow. It imports Firebase to read the email of the logged-in user, validates the forms, builds the review panel, creates the order and sends the confirmation email.

**Table 18: Functions in checkout.js**

| Function                | Description                                                             |
|-------------------------|-------------------------------------------------------------------------|
| goToStep(n)             | Validates the current step, shows step n and updates the progress bar   |
| validateStep1()         | Checks address fields, 10-digit phone and 6-digit PIN                   |
| validateStep2()         | Checks UPI ID or card details according to the payment option           |
| populateReview()        | Fills the review panel in step 3                                        |
| renderCheckoutSummary() | Draws the item summary beside the form                                  |
| placeOrder()            | Creates the order, saves it, clears the cart, sends email and redirects |

| Function                | Description                                      |
|-------------------------|--------------------------------------------------|
| sendConfirmationEmail() | Calls emailjs.send() with the order details      |
| getCurrentUserEmail()   | Returns the logged-in user's email from Firebase |

#### 5.10.4 Order ID and Delivery Date

The order ID is created by joining the letters "BYT" with the last eight digits of the current time in milliseconds, for example BYT12345678. The expected delivery date is calculated by adding seven days to the current date and formatting it in the Indian English style, for example "1 July 2026".

```
const orderId = "BYT" + Date.now().toString().slice(-8);
const d = new Date();
d.setDate(d.getDate() + 7);
const deliveryDate = d.toLocaleDateString("en-IN",
  { day: "numeric", month: "long", year: "numeric" });
```

## 6. Authentication Using Firebase

### 6.1 Why Firebase Authentication

Building a safe login system normally requires a server, a database of users, password hashing, session management and protection against attacks. Firebase Authentication provides all this as a service. The project only needs to include the Firebase JavaScript library and call a few functions. Passwords are never visible to the project code after they are submitted and are stored and protected by Google.

### 6.2 Authentication Methods

Table 19: Firebase methods used

| Method                                 | Used for                                             |
|----------------------------------------|------------------------------------------------------|
| initializeApp()                        | Connects the website to the Firebase project         |
| getAuth()                              | Gets the authentication service object               |
| createUserWithEmailAndPassword()       | Registering a new account                            |
| signInWithEmailAndPassword()           | Logging in an existing user                          |
| GoogleAuthProvider + signInWithPopup() | Google sign-in through a pop-up window               |
| onAuthStateChanged()                   | Detecting login or logout and updating the interface |
| signOut()                              | Logging the user out                                 |

### 6.3 Login Flow

- The user enters email and password and presses Login.
- auth.js calls signInWithEmailAndPassword() with the entered values.
- Firebase verifies the credentials on its servers.
- On success Firebase returns a user object; onAuthStateChanged() fires.
- The navigation bar is updated and the user is redirected to the home page.
- On failure an error code is converted to a friendly message and shown on the form.

## 6.4 Google Sign-In

When "Sign in with Google" is pressed, `signInWithPopup()` opens the Google account chooser. The user selects an account and approves access. The complete OAuth process is handled by Firebase, so the project does not manage tokens or client secrets.

## 6.5 Session Persistence

Firebase keeps the user signed in even after the tab or browser is closed. The session is stored by the Firebase library in the browser (IndexedDB). On the next visit the library restores the session and `onAuthStateChanged()` runs again, so the user stays logged in without entering the password again.

## 6.6 Protected Pages

Four pages require login: `checkout.html`, `orders.html`, `order-confirmation.html` and `account.html`. When the authentication listener finds no logged-in user on one of these pages, it redirects to `login.html`. This is a convenience control for the interface; real protection of data would also require rules on a server, as discussed in Chapter 10.

## 6.7 Error Messages

**Table 20: Friendly error messages**

| Firestore error code        | Message shown to the user                   |
|-----------------------------|---------------------------------------------|
| auth/invalid-email          | Please enter a valid email address.         |
| auth/user-not-found         | No account found with this email.           |
| auth/wrong-password         | Incorrect password. Please try again.       |
| auth/email-already-in-use   | An account with this email already exists.  |
| auth/weak-password          | Password should be at least 6 characters.   |
| auth/too-many-requests      | Too many attempts. Please try again later.  |
| auth/network-request-failed | Network error. Check your connection.       |
| auth/popup-closed-by-user   | Sign-in popup was closed. Please try again. |
| Any other code              | Something went wrong. Please try again.     |

# 7. Email Notification System

---

## 7.1 About EmailJS

EmailJS is a service that sends emails from client-side code. The developer connects an email service such as Gmail, designs a template on the EmailJS dashboard and then calls a JavaScript function from the website with the values to be placed in the template. This avoids the need for a custom mail server.

## 7.2 How It Works in Buyit

- The customer clicks "Place Order" in step 3 of the checkout.
- `checkout.js` calls `getCurrentUserEmail()` to read the email address from Firebase.
- It prepares a `templateParams` object containing all the order details.
- It calls `emailjs.send()` with the service ID, template ID and the parameters.
- EmailJS fills the template and sends the message.

- The customer receives an HTML email with the order summary.

## 7.3 Template Variables

**Table 21: EmailJS template variables**

| Variable             | Value                                      |
|----------------------|--------------------------------------------|
| {{to_email}}         | Email address of the logged-in user        |
| {{to_name}}          | Full name entered in checkout step 1       |
| {{order_id}}         | Generated order ID such as BYT12345678     |
| {{order_date}}       | Date of the order                          |
| {{delivery_date}}    | Order date plus seven days                 |
| {{payment_method}}   | Cash on Delivery, UPI or Credit/Debit Card |
| {{delivery_address}} | Full address from the checkout form        |
| {{order_items}}      | List of items with quantity and price      |
| {{order_total}}      | Total amount in Indian Rupees              |

## 7.4 Handling Failures

If sending the email fails because of a network problem or because the monthly limit has been reached, the order is still treated as successful. The error is written to the browser console and the customer is not blocked. This design follows the principle that a secondary service must not stop the main business process. The free EmailJS plan allows 200 emails per month, which is sufficient for a prototype.

# 8. User Interface Design

A consistent and clean interface makes a shopping website easy to use. All pages of Buyit share a single stylesheet so that colours, buttons, cards and spacing look the same everywhere.

## 8.1 Colour Palette

Colours are defined once as CSS variables and reused across the stylesheet. Changing one value changes the colour of the whole site.

**Table 22: Colour variables**

| Variable        | Hex     | Used for                                    |
|-----------------|---------|---------------------------------------------|
| --primary-color | #006A5B | Buttons, links, accents, active states      |
| --primary-light | #e6f4f1 | Card backgrounds, hover states, info boxes  |
| --dark-bg       | #110d1b | Navigation bar, footer                      |
| --light-bg      | #f4fafa | Page and section backgrounds                |
| --accent-orange | #fb641b | Buy Now and checkout call-to-action buttons |
| --text-dark     | #333333 | Main body text                              |
| --text-light    | #ffffff | Text on dark backgrounds                    |
| --danger        | #e74c3c | Cancel, remove and error elements           |

## 8.2 Typography

The Inter font family from Google Fonts is used throughout. It is clean and readable at small and large sizes. Font sizes use the rem unit so that text scales properly with the user's browser settings.

## 8.3 Layout Techniques

- **Navigation bar:** Flexbox with space-between alignment.
- **Product grid:** CSS Grid with auto-fill and minimum column width of 250 pixels.
- **Category grid:** CSS Grid with auto-fit for even distribution of cards.
- **Cart and checkout:** Flex rows that wrap into a single column on small screens.

## 8.4 Responsive Design

The stylesheet contains media queries at 768 pixels and 480 pixels. At tablet and mobile widths the multi-column layouts change to a single column, the navigation bar stacks vertically, the search bar takes the full width, and the cart and checkout panels are placed one below the other. The design is intended to adapt to common phone and tablet sizes, and it can be checked using the browser's device mode.

## 8.5 Interface Feedback

- **Toast message:** confirms that an item was added to the cart without interrupting the user.
- **Error highlighting:** invalid form fields receive a red border and a message.
- **Progress indicator:** shows which checkout step is active and which are complete.
- **Loading state:** the Place Order button shows a spinner while the order is being processed.
- **Confirmation modal:** asks the user to confirm before an order is cancelled.
- **Status badges:** colour and icon indicate the state of each order.

# 9. Testing

---

## 9.1 Testing Approach

Testing was carried out manually using the browser. Each feature was tried with correct input, with incorrect input and with unusual actions such as refreshing a page in the middle of a process. The browser developer tools were used to inspect localStorage and sessionStorage, to watch network requests to Firebase and EmailJS, and to check the layout in mobile view.

The types of testing applied were:

- **Functional testing** – checking that each feature behaves as required.
- **Validation testing** – entering wrong or missing values in forms.
- **Usability testing** – checking whether a first-time user can complete a purchase without help.
- **Responsive testing** – viewing the pages at different screen widths.
- **Compatibility testing** – opening the site in different browsers.

## 9.2 Test Cases

The table below lists the main test cases that describe the manual test plan for the application, together with the expected behaviour.

**Table 23: Test cases**

| ID  | Scenario                 | Action / Input                            | Expected result                                      |
|-----|--------------------------|-------------------------------------------|------------------------------------------------------|
| T1  | Add to cart              | Click Add to Cart on a product            | Item saved in cart; toast message shown              |
| T2  | Add same item twice      | Click Add to Cart again                   | Quantity increases to 2; no duplicate line           |
| T3  | Change quantity          | Press + or - on cart page                 | Quantity and total update immediately                |
| T4  | Remove item              | Press Remove                              | Item disappears; total recalculated                  |
| T5  | Empty cart               | Remove all items                          | Empty-cart message; checkout button disabled         |
| T6  | Cart persistence         | Refresh the page after adding items       | Cart contents remain                                 |
| T7  | Search match             | Search "lenovo"                           | Only the Lenovo card is shown                        |
| T8  | Search no match          | Search "xyz"                              | "No results found" message displayed                 |
| T9  | Buy Now                  | Click Buy Now on a product                | Cart holds only that item; checkout opens            |
| T10 | Valid registration       | New email and password of 6+ characters   | Account created; user logged in                      |
| T11 | Duplicate registration   | Register an existing email                | Message: account already exists                      |
| T12 | Wrong password           | Correct email, wrong password             | Friendly error message shown                         |
| T13 | Google sign-in           | Choose a Google account                   | User logged in; username appears in navbar           |
| T14 | Protected page           | Open checkout.html without login          | Redirected to login.html                             |
| T15 | Empty address field      | Leave a field blank in step 1             | Field highlighted; message shown; step blocked       |
| T16 | Invalid phone            | Enter 9-digit phone                       | Error: enter valid 10-digit number                   |
| T17 | Invalid PIN              | Enter 5-digit PIN                         | Error: enter valid 6-digit PIN                       |
| T18 | Invalid UPI              | Enter UPI ID without "@"                  | Error: enter valid UPI ID                            |
| T19 | Incomplete card          | Enter short card number                   | Error: fill in card details correctly                |
| T20 | Place order              | Complete all steps with valid data        | Order saved; cart cleared; confirmation page shown   |
| T21 | Confirmation email       | After placing order                       | Email arrives at the logged-in user's address        |
| T22 | Direct confirmation page | Open order-confirmation.html in a new tab | "No recent order found" message                      |
| T23 | Order history            | Open orders.html                          | Orders listed newest first with status badge         |
| T24 | Cancel order             | Cancel a Placed order and confirm         | Status changes to Cancelled; button disappears       |
| T25 | Logout                   | Press Logout on account page              | Session ended; home page shown; Login button visible |
| T26 | Mobile layout            | View at 375 px width                      | Single-column layout; no horizontal scrolling        |

## 9.3 Observations

During testing the team noted several points that guided improvements: the use of a toast message instead of a browser alert for a smoother experience; reading the user's email directly from Firebase in checkout.js instead of from a page element that exists only on the account page; and treating email failure as non-fatal so that orders are never lost because of a messaging problem.

## 10. Security Considerations

---

Buyit is a prototype that runs completely in the browser. Security was considered in the design, but a pure frontend application cannot offer the same protection as a system with a server. This chapter explains what is protected, what is not, and what should be done before a real launch.

### 10.1 What Is Handled Securely

- Passwords are handled and stored by Firebase; they are never saved in the project code or in browser storage.
- Google sign-in uses the standard OAuth process managed by Google; no client secret is placed in the JavaScript.
- Firebase keeps its session tokens in its own browser storage managed by the library.
- Card details typed at checkout are not sent to any server and are not stored.

### 10.2 Points to Improve

**Table 24: Security observations and recommendations**

| Area               | Current situation                                                                                                               | Recommendation                                                                                         |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Configuration keys | Firebase configuration and EmailJS identifiers are written in the JavaScript files; they are public by nature in a frontend app | Restrict the Firebase API key and EmailJS usage to the website's domain from the respective dashboards |
| Order data         | Orders are stored in localStorage and can be viewed or changed by anyone using the same browser                                 | Store orders in a cloud database such as Firestore with per-user security rules                        |
| Payment            | No real payment is processed; card fields are for demonstration only                                                            | Use a certified gateway such as Razorpay or Stripe; never handle raw card data                         |
| Input handling     | Form values and product names are placed into page HTML without sanitising                                                      | Sanitise or escape all user-supplied text before displaying it                                         |
| Page protection    | Redirects for protected pages run in the browser and can be bypassed by a skilled user                                          | Enforce access on a server or through database security rules                                          |
| Transport security | HTTPS depends on the host                                                                                                       | Deploy on a host that enforces HTTPS, such as Netlify, Vercel or Firebase Hosting                      |

It is important to state clearly that the card number, expiry date and CVV fields of the checkout page exist only to demonstrate the payment step. They must be replaced by a certified payment gateway before the project is used for real transactions.

## 11. Limitations and Future Scope

---

### 11.1 Current Limitations

**Table 25: Limitations of version 1.0**

| Limitation                                          | Reason                                               | Impact                                                  |
|-----------------------------------------------------|------------------------------------------------------|---------------------------------------------------------|
| Orders and cart are lost if browser data is cleared | Data is kept only in localStorage                    | Order history disappears                                |
| Cart and orders are not shared between devices      | localStorage is specific to a browser                | Different data on phone and laptop                      |
| No real payment processing                          | No payment gateway is integrated                     | Cannot accept actual money                              |
| Order status never changes automatically            | No backend to update status                          | Orders remain "Placed"                                  |
| No administrator panel                              | Application is frontend only                         | Products and orders cannot be managed from an interface |
| Only five products                                  | Products are written in HTML                         | New products need code changes                          |
| Some categories have no products                    | Watches, Headphones and Books cards are placeholders | Those links lead to no items                            |
| Email limit of 200 per month                        | EmailJS free plan                                    | Emails stop after the limit is reached                  |

### 11.2 Future Scope

- Store orders and carts in Firebase Firestore so that they are available on every device.
- Integrate a payment gateway such as Razorpay or Stripe for online payments.
- Develop an admin dashboard to add and edit products and to update order status.
- Move the product list into a database so that new products can be added without editing HTML.
- Add a wishlist, product reviews and ratings.
- Add image zoom on product pages.
- Add an address book so that saved addresses can be reused.
- Send a cancellation email using a second EmailJS template.
- Add an order tracking timeline showing each stage of delivery.
- Deploy the project on a free host such as Netlify or Firebase Hosting to make it publicly available.
- Add a backend with Node.js or a similar technology for secure payment verification and personalised recommendations.

## 12. Conclusion

---

### 12.1 Summary

Buyit – In Your Way successfully demonstrates a complete online shopping experience using only HTML, CSS and JavaScript together with two external services. A customer can search and browse products, read details, manage a cart, log in with email or Google, complete a validated three-step checkout, receive a confirmation email and later view or cancel the order. The interface is consistent,

responsive and easy to use, and the code is organised into small modules that can be understood and extended.

The project meets the objectives set at the beginning: a working storefront, persistent cart, secure authentication, validated checkout, email notification and order history. Its limitations, mainly the absence of a backend and of real payments, are clearly identified, and a practical route for future development is described.

## 12.2 Learning Outcomes

- Writing structured, semantic HTML and reusable CSS using variables, Flexbox, Grid and media queries.
- Using JavaScript to manipulate the DOM, handle events, validate forms and work with JSON.
- Managing application state with localStorage and sessionStorage.
- Integrating third-party services: Firebase Authentication and EmailJS.
- Understanding authentication flow, session persistence and protection of pages.
- Planning pages, data structures and user flows before coding.
- Testing, debugging with browser developer tools and handling errors gracefully.
- Using Git and GitHub for collaboration between two team members.
- Preparing technical documentation for a software project.
- Awareness of security issues in web applications and the difference between a prototype and a production system.

## 12.3 Team Collaboration

The project was developed by a team of two students. The work was shared between pages and modules, and the code was combined through GitHub. Working as a team taught the importance of agreeing on file names, naming conventions and a common design before starting, as well as reviewing each other's work and solving integration problems together.

## 12.4 Acknowledgement of Support

The team is grateful to INTERNbee for providing the internship opportunity and guidance, and to the faculty of Government Polytechnic, Raghapur, Supaul for their support throughout the project.

## References

---

- MDN Web Docs – HTML, CSS and JavaScript reference. <https://developer.mozilla.org>
- MDN Web Docs – Web Storage API (localStorage and sessionStorage). [https://developer.mozilla.org/en-US/docs/Web/API/Web\\_Storage\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API)
- Firebase Documentation – Authentication. <https://firebase.google.com/docs/auth>
- EmailJS Documentation. <https://www.emailjs.com/docs>
- Font Awesome Documentation. <https://fontawesome.com>
- Google Fonts – Inter. <https://fonts.google.com/specimen/Inter>
- W3Schools – HTML, CSS and JavaScript tutorials. <https://www.w3schools.com>
- Git and GitHub Documentation. <https://docs.github.com>

## Appendix A: Running the Project

---

### A.1 Running Locally

- Download or clone the project folder (Buyit-main) from GitHub.
- Open the folder in Visual Studio Code.
- Open index.html in a browser. For ES modules and Firebase login to work reliably, use a local server such as the VS Code "Live Server" extension instead of opening the file directly.
- Make sure the computer is connected to the internet so that Firebase, EmailJS, fonts and icons can load.

### A.2 Setting Up Firebase

- Create a project in the Firebase console.
- Enable the Email/Password and Google sign-in methods under Authentication.
- Add the website's domain (for example localhost and the hosting domain) to the authorised domains list.
- Copy the web app configuration into the firebaseConfig object in auth.js and checkout.js.

### A.3 Setting Up EmailJS

- Create an EmailJS account and connect an email service.
- Create a template that uses the variables listed in the template variables table of Chapter 7.
- Place the service ID, template ID and public key in checkout.js.

### A.4 Deployment

Because Buyit is a static site, it can be deployed by uploading the folder to Netlify, Vercel, GitHub Pages or Firebase Hosting. After deployment, add the new domain to Firebase authorised domains and restrict the API keys to that domain.

## Appendix B: Glossary

---

**Table 26: Glossary of terms**

| Term           | Meaning                                                                    |
|----------------|----------------------------------------------------------------------------|
| API            | Application Programming Interface; a way for programs to communicate       |
| CDN            | Content Delivery Network; servers that deliver libraries and fonts quickly |
| DOM            | Document Object Model; the browser's representation of a web page          |
| ES Module      | A JavaScript file that uses import and export                              |
| JSON           | JavaScript Object Notation; a text format for storing data                 |
| localStorage   | Browser storage that remains until deleted                                 |
| sessionStorage | Browser storage that lasts only for the open tab                           |
| OAuth          | A standard method for logging in with another provider such as Google      |
| Toast          | A small message that appears briefly on screen                             |
| UPI            | Unified Payments Interface; an instant payment system in India             |